GhostAudit — Software Requirements Document (SRD) & Project Blueprint

Track 2: Gemma Financial Compliance & Risk Triage — Build with Gemma: Bengaluru AI Sprint 2026

---

1. Project Overview

Project Name: GhostAudit — AI Fraud Pattern & Compliance Risk Triage Copilot

Vision: Give SMEs and small review teams the same explainable, investigator-grade compliance intelligence that large BFSI institutions have, but in a self-serve, plain-language, one-day-buildable product.

Problem Statement: Indian SMEs and small review/compliance teams face two compounding problems. First, real fraud patterns (split invoicing to bypass approval thresholds, ghost/shell vendors, delayed-payment MSME Act violations) frequently go undetected until an external audit, bank review, or regulator flags them — at which point penalties, blocked credit lines, or reputational damage have already occurred. Second, the compliance tools that do exist (Perfios KScan AI, Inscribe, Solytics, Thomson Reuters CLEAR) are enterprise-priced, built for BFSI risk teams, and output static risk scores or alert floods rather than explainable reasoning — leaving SME owners and smaller reviewers with either no tooling or tooling that causes alert fatigue without context.

Why This Matters: SMEs contribute significantly to India's economy but operate with thin compliance margins. A single missed GST mismatch or undetected ghost vendor can trigger blocked input tax credit, loan rejection, or regulatory penalty. Existing tools solve this for large enterprises, not for the SME itself or the analyst reviewing a handful of SME accounts.

Existing Limitations: - Static rule-based systems flag "this looks unusual" without explaining why. - No competing tool frames the SME as the beneficiary of self-audit rather than the subject being screened. - Enterprise GRC pricing (Rs 25-80 lakh/year) is inaccessible to SMEs and small teams. - No plain-language, non-expert-friendly explanation layer exists in the market.

Proposed Solution: GhostAudit combines a deterministic rule-based detection engine (split invoice / round-tripping detection, ghost vendor verification) with Gemma acting as a reasoning and communication layer. Gemma investigates each flagged case using an Evidence to Defense to Verdict structure (AI Prosecutor), produces a step-by-step reasoning trail (AI Risk Replay), explains findings in plain language for non-experts, and compiles confirmed risks into an audit-ready report.

Target Users: - SME owners self-checking their books before a bank/audit review. - Compliance/review teams and accountants needing faster, less noisy triage. - Auditors and lenders evaluating SME risk who want evidence-backed explanations, not raw scores.

User Personas: 1. Priya, SME Owner - runs a 15-person trading business, has no in-house compliance officer, needs plain-language alerts before a loan renewal. 2. Arun, Junior Compliance Analyst - reviews 40+ SME accounts weekly, suffers alert fatigue from static rule engines, needs prioritized, explained cases. 3. Meena, External Auditor - needs a clean, exportable report summarizing confirmed risks instead of raw transaction logs.

Key Objectives: - Detect at least 2 concrete, real fraud patterns reliably on mock data. - Provide reasoning-based explanations, not just scores. - Produce an audit-ready report as tangible output. - Ship a working, demoable product within a one-day hackathon window.

Expected Impact: Reduced reviewer alert fatigue, faster SME self-checks before external audits, and a proof-of-concept for accessible, explainable compliance intelligence for the underserved SME segment.

---

2. Complete Feature List (All 43 Features - Tiered)

Each feature: Description | Why it exists | User benefit | Complexity | Dependencies | Priority

TIER 1 - MUST BUILD (Hackathon Core)

#	Feature	Description	Why It Exists	User Benefit	Complexity	Dependencies	Priority
1	Split Invoice / Round-Tripping Detection	Flags invoices broken into smaller amounts or circular vendor-account transfers to bypass approval thresholds	Real, common SME fraud pattern	Concrete, explainable risk detection	Medium	Mock transaction dataset, rules engine	P0
2	Ghost Vendor Verification	Cross-checks vendor GSTIN format, address, bank account patterns to flag shell vendors	Fake vendors are a leading fraud vector	Prevents payments to non-existent entities	Low-Medium	Vendor dataset, rules engine	P0
3	AI Risk Replay	Step-by-step reasoning trail (Step 1-2-3-Verdict) instead of static score	Differentiator vs. static rule tools	Builds trust via transparency	Medium	Detection engine output, Gemma API	P0
4	AI Prosecutor (Evidence -> Defense -> Verdict)	Gemma builds a case: evidence for risk, plausible innocent explanation, final verdict	Reframes detection as investigation, most unique feature	Reduces false-positive fatigue, feels like a real analyst	Medium-High	Risk Replay output, Gemma API	P0
5	Reviewer Compliance Copilot / Triage Queue	Ranks flagged cases by severity/ confidence for reviewer action	Makes tool usable as a workflow, not a one-off script	Reviewers act on priority, not noise	Low-Medium	Detection engine, verdict data	P0
6	Explain Like I'm Not an Auditor (Plain-Language Explanations)	Converts technical flags into plain language for non-expert SME owners	SME owners lack financial/ compliance jargon fluency	Makes findings actionable for non-experts	Low	Gemma API, verdict data	P0
7	Audit-Ready Report Generator	Aggregates confirmed risks into a structured, exportable report	Gives a tangible final output for banks/auditors	Turns detection into a usable deliverable	Low-Medium	All above features	P0

TIER 2 - SHOULD BUILD (If Time Allows)

#	Feature	Description	Why It Exists	User Benefit	Complexity	Dependencies	Priority
8	Timeline Investigation	Reconstructs sequence of financial events per case	Adds temporal context to flagged cases	Easier to spot patterns over time	Low-Medium	Transaction dataset	P1
9	Financial Crime Storyboard	Presents investigation as a narrative sequence	Improves demo clarity for judges	Non-technical stakeholders understand cases faster	Low	AI Prosecutor output	P1
10	Risk / Relationship Graph	Visualizes links between vendors, accounts, invoices	Graph-based fraud detection is a proven, credible technique	Visual pattern discovery (fraud rings)	Medium	Vendor + transaction data	P1
11		Verifies vendor GST details against		Higher detection credibility	Low-Medium	Vendor dataset	P1

#	Feature	Description	Why It Exists	User Benefit	Complexity	Dependencies	Priority
	GST Registry Cross-Verification	mock registry records	Adds a second, regulator-relevant compliance check				
12	Vendor Risk Profiling	Dynamic risk score per vendor based on accumulated evidence	Extends single-case detection to vendor-level trust	Reviewers can deprioritize/ blacklist vendors	Medium	Ghost Vendor Verification	P1

TIER 3 - NICE TO HAVE / FUTURE ROADMAP

#	Feature	Description	Complexity	Priority
13	Cross-Document Contradiction Detection	Compares onboarding docs, invoices, bank records for inconsistencies	High	P2
14	Compliance Memory	Remembers previous investigations and recurring risks across sessions	Medium-High	P2
15	Reverse Audit	Predicts where fraud/compliance failures are most likely before they occur	High	P2
16	Cross-Document Investigation Engine	Full multi-document correlation engine	High	P2
17	Investigator Behavior Cloning	Learns to prioritize cases like an experienced analyst	High	P2
18	Compliance DNA	Learns a company's normal financial "fingerprint"	Medium-High	P2
19	Peer Benchmark Anomaly Detection	Compares an SME against similar businesses	Medium	P2
20	Compliance Drift Detection	Identifies gradual compliance deterioration over time	Medium	P2
21	Financial Weather Forecast	Predicts future compliance risk trends	Medium	P2
22	Invisible Employee	Background AI continuously monitors transactions	Medium	P2
23	Behavioral Pattern Monitoring	Detects unusual behavior changes over time	Medium	P2
24	Financial Digital Twin	Live contextual model of the business	High	P2
25	Behavioral Compliance Intelligence Engine	Combines behavioral, document, and transaction analysis	High	P2
26	Multi-Regulator Correlation / Dashboard	Correlates findings across GST, banking, KYC	Medium-High	P2
27	Multi-Source Evidence Correlation	Connects transactions, docs, emails, invoices into one view	High	P2
28	Interactive Investigation Workspace	Case-based investigation UI instead of single-file analysis	Medium-High	P2
29	AI Financial Investigation Workspace	Full investigator workbench (superset of above)	High	P2
30	SME Self-Audit Copilot	Standalone self-check mode for SME owners	Low-Medium	P2
31	Delayed Payment Risk & Recovery Agent	Flags MSME 45-day payment rule violations, drafts recovery notice	Medium	P2
32	DPDP Readiness Checker	Assesses data protection compliance gaps	Low-Medium	P2
33	MSME Formalization Coach	Guides informal businesses toward formal compliance	Low-Medium	P2
34	Regulatory Shock Simulator	Simulates impact of new regulations on records	Medium	P2
35	Advertising Compliance Auditor (Ad-Shield)	Checks marketing claims against compliance rules	Medium	P3
36	Dark Pattern Financial App Auditor	Detects manipulative UX/financial patterns in apps	High	P3
37	Property / Collateral Loan Auditor	Verifies collateral documentation for loans	Medium	P3

#	Feature	Description	Complexity	Priority
38	Cross-Border Customs / HSN Compliance	Validates import/export HSN codes	High	P3
39	WhatsApp / Email Financial Compliance	Analyzes informal-channel business communication	Medium	P2
40	Verbal vs Written Contract Auditor	Compares contracts against communication transcripts	Medium	P2
41	Shadow Procurement Detector	Detects employees bypassing procurement controls	Medium	P2
42	Auto-Remediation Agent	Suggests corrective actions for confirmed risks	Low-Medium	P2
43	AI Negotiator	Drafts clarification emails to vendors	Low	P2

3. Feature Dependency Graph

```
Auth (Login/Session)
   |
Database + Mock Dataset (Transactions, Vendors)
   |
Backend Setup (Express API)
   |
Detection Engine (Split Invoice + Ghost Vendor)
   |
Risk Scoring Output (JSON per case)
   |
AI Risk Replay (Gemma reasoning trail)
   |
AI Prosecutor (Evidence -> Defense -> Verdict)
   |
Reviewer Triage Queue (ranks verdicts)
   |
Plain-Language Explanation (Gemma)
   |
Audit-Ready Report Generator
   |
[Tier 2] Timeline / Storyboard / Risk Graph / GST Verify / Vendor Profiling
   |
[Tier 3] Roadmap Features (post-hackathon)
```

4. Feature Development Order (Chunks)

Chunk 1 - Foundation - Project scaffolding (frontend + backend repos) - SQLite schema + mock dataset seeding - Express server setup, base routes - React app shell, routing, base layout

Chunk 2 - Detection Engine - Split invoice / round-tripping rule logic - Ghost vendor verification rule logic - Risk scoring JSON structure - API endpoints for detection results

Chunk 3 - AI Reasoning Layer - Gemma API integration (Python microservice) - AI Risk Replay prompt + response parsing - AI Prosecutor prompt (Evidence/Defense/Verdict) - Plain-language explanation prompt

Chunk 4 - Frontend Workflow - Dashboard: flagged case list - Triage queue: sorted by severity/confidence - Case detail view: reasoning trail + verdict - Report export view

Chunk 5 - Report & Polish - Audit-ready report generation (Gemma + template) - Export to PDF/print view - UI polish, error/empty/loading states - End-to-end demo run-through

Chunk 6 - Stretch (Tier 2, if time remains) - Timeline Investigation - Risk / Relationship Graph - GST Registry Cross-Verification - Vendor Risk Profiling

5. Technology Stack (Strict)

Frontend: HTML, CSS, JavaScript, React.js (Create React App or Vite with JS templates - no TypeScript)

Backend: Node.js, Express.js (JavaScript only)

AI Layer: Python microservice (Flask/FastAPI acceptable) calling Gemma API; Node backend communicates with Python service via internal REST calls

Database: SQL - SQLite for hackathon (file-based, zero-config), schema designed to be Postgres-compatible for future scaling

Explicitly Excluded: TypeScript, Next.js, NestJS, MongoDB

---

6. Complete Folder Structure

```
ghostaudit/
├── frontend/
│   ├── public/
│   │   └── index.html
│   ├── src/
│   │   ├── assets/
│   │   │   ├── images/
│   │   │   └── icons/
│   │   ├── components/
│   │   │   ├── common/          (Button, Card, Badge, Loader, EmptyState)
│   │   │   ├── dashboard/       (CaseList, TriageQueue, KPIWidget)
│   │   │   ├── case/            (RiskReplayTrail, ProsecutorVerdict, ExplanationPanel)
│   │   │   └── report/          (ReportPreview, ExportButton)
│   │   ├── pages/
│   │   │   ├── DashboardPage.jsx
│   │   │   ├── CaseDetailPage.jsx
│   │   │   ├── ReportPage.jsx
│   │   │   └── LoginPage.jsx
│   │   ├── routes/
│   │   │   └── AppRoutes.jsx
│   │   ├── services/
│   │   │   └── apiClient.js
│   │   ├── state/
│   │   │   └── AppContext.js    (React Context, no Redux needed for hackathon scope)
│   │   ├── utils/
│   │   │   └── formatters.js
│   │   ├── constants/
│   │   │   └── riskLevels.js
│   │   ├── App.jsx
│   │   └── index.js
│   └── package.json
│
├── backend/
│   ├── src/
│   │   ├── config/
│   │   │   └── db.config.js
│   │   ├── routes/
│   │   │   ├── auth.routes.js
│   │   │   ├── transactions.routes.js
│   │   │   ├── vendors.routes.js
│   │   │   ├── cases.routes.js
│   │   │   └── reports.routes.js
│   │   ├── controllers/
│   │   │   ├── auth.controller.js
│   │   │   ├── case.controller.js
│   │   │   └── report.controller.js
│   │   ├── services/
│   │   │   ├── detection.service.js     (split invoice + ghost vendor rules)
│   │   │   ├── riskScoring.service.js
│   │   │   ├── gemmaClient.service.js   (calls Python AI service)
│   │   │   └── report.service.js
```

```
│   │   ├── middlewares/
│   │   │   ├── auth.middleware.js
│   │   │   ├── errorHandler.middleware.js
│   │   │   └── validate.middleware.js
│   │   ├── models/
│   │   │   ├── transaction.model.js
│   │   │   ├── vendor.model.js
│   │   │   └── case.model.js
│   │   ├── utils/
│   │   │   └── logger.js
│   │   └── app.js
│   ├── server.js
│   └── package.json
│
├── ai-service/                          (Python)
│   ├── prompts/
│   │   ├── risk_replay_prompt.py
│   │   ├── prosecutor_prompt.py
│   │   ├── explain_prompt.py
│   │   └── report_prompt.py
│   ├── services/
│   │   ├── gemma_service.py
│   │   ├── reasoning_pipeline.py
│   │   └── report_generator.py
│   ├── main.py                          (FastAPI/Flask entrypoint)
│   └── requirements.txt
│
├── database/
│   ├── schema.sql
│   ├── seed_data.sql
│   └── ghostaudit.db                    (SQLite file, generated)
│
├── docs/
│   ├── SRD.md                           (this document)
│   ├── API_CONTRACTS.md
│   └── PITCH_NOTES.md
│
├── tests/
│   ├── backend/
│   ├── ai-service/
│   └── frontend/
│
├── .env.example
├── .gitignore
└── README.md
```

---

7. API Architecture

Endpoint	Method	Purpose	Input	Output	Frontend Page	AI Module
/api/auth/login	POST	Reviewer login	email, password	token, user	LoginPage	-
/api/transactions	GET	Fetch mock transaction dataset	query params (filters)	list of transactions	DashboardPage	-
/api/vendors	GET	Fetch mock vendor dataset	query params	list of vendors	DashboardPage	-
/api/cases/scan	POST	Run detection engine (split invoice + ghost vendor)	datasetId	list of case + riskScore	DashboardPage	detection.service.js
/api/cases	GET		query params (sort, filter)	list of case	DashboardPage	-

Endpoint	Method	Purpose	Input	Output	Frontend Page	AI Module
		List flagged cases (triage queue)				
/api/cases/:id	GET	Case detail	case id	case, riskReplay, verdict	CaseDetailPage	gemmaClient -> reasoning_pipeline.py
/api/cases/:id/replay	POST	Generate AI Risk Replay trail	case id	steps array	CaseDetailPage	risk_replay_prompt.py
/api/cases/:id/verdict	POST	Generate AI Prosecutor verdict	case id	evidence, defense, verdict	CaseDetailPage	prosecutor_prompt.py
/api/cases/:id/explain	POST	Plain-language explanation	case id	explanationText	CaseDetailPage	explain_prompt.py
/api/cases/:id/feedback	POST	Reviewer confirms/ overturns verdict	verdictOutcome	updatedCase	CaseDetailPage	-
/api/reports/generate	POST	Compile audit-ready report	caseIds array	reportUrl / reportJson	ReportPage	report_generator.py
/api/reports/:id/export	GET	Export report as PDF/text	report id	file stream	ReportPage	-

Error format (standard across all endpoints):

```
{ "error": true, "code": "CASE_NOT_FOUND", "message": "Case with given ID does not exist" }
```

---

8. Database Design (SQLite)

Tables:

```
-- vendors
CREATE TABLE vendors (
  vendor_id INTEGER PRIMARY KEY AUTOINCREMENT,
  vendor_name TEXT NOT NULL,
  gstin TEXT,
  address TEXT,
  bank_account TEXT,
  is_verified BOOLEAN DEFAULT 0,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- transactions
CREATE TABLE transactions (
  transaction_id INTEGER PRIMARY KEY AUTOINCREMENT,
  vendor_id INTEGER NOT NULL,
  amount REAL NOT NULL,
  transaction_date DATETIME NOT NULL,
  invoice_number TEXT,
  payment_method TEXT,
  approval_threshold REAL,
  FOREIGN KEY (vendor_id) REFERENCES vendors(vendor_id)
);

-- cases (flagged risk items)
CREATE TABLE cases (
  case_id INTEGER PRIMARY KEY AUTOINCREMENT,
  transaction_id INTEGER,
  vendor_id INTEGER,
  case_type TEXT CHECK(case_type IN ('SPLIT_INVOICE','GHOST_VENDOR')),
```

```
  risk_score TEXT CHECK(risk_score IN ('LOW','MEDIUM','HIGH')),
  reasoning_trail TEXT,       -- JSON string of Risk Replay steps
  evidence TEXT,               -- JSON string
  defense TEXT,
  verdict TEXT,
  plain_explanation TEXT,
  reviewer_feedback TEXT CHECK(reviewer_feedback IN ('CONFIRMED','OVERTURNED', NULL)),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (transaction_id) REFERENCES transactions(transaction_id),
  FOREIGN KEY (vendor_id) REFERENCES vendors(vendor_id)
);

-- reports
CREATE TABLE reports (
  report_id INTEGER PRIMARY KEY AUTOINCREMENT,
  case_ids TEXT,               -- JSON array of case_id
  report_content TEXT,
  generated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- reviewers (auth)
CREATE TABLE reviewers (
  reviewer_id INTEGER PRIMARY KEY AUTOINCREMENT,
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  name TEXT
);
```

Indexes:

```
CREATE INDEX idx_cases_risk_score ON cases(risk_score);
CREATE INDEX idx_transactions_vendor ON transactions(vendor_id);
```

Example Records:

```
{ "vendor_id": 1, "vendor_name": "Sri Lakshmi Traders", "gstin": "29ABCDE1234F1Z5", "is_verified": 0 }
{ "transaction_id": 101, "vendor_id": 1, "amount": 49000, "invoice_number": "INV-2201", "approval_threshold": 50000 }
{ "case_id": 1, "transaction_id": 101, "case_type": "SPLIT_INVOICE", "risk_score": "HIGH" }
```

---

9. AI Architecture

Role of Gemma: Gemma serves as the reasoning engine operating on top of a deterministic anomaly-detection layer. The rules engine (Node.js) flags transactions/vendors; Gemma investigates, explains, and reports.

Pipeline: 1. Detection engine flags a case -> structured JSON (case_type, transaction data, threshold breach details). 2. **Risk Replay prompt**: Gemma receives structured flag data, returns a numbered reasoning trail (Step 1 -> Step 2 -> Step 3 -> Risk Score). 3. **Prosecutor prompt**: Gemma receives the reasoning trail, generates Evidence (why it looks risky), Defense (most plausible innocent explanation), and Verdict. 4. **Explain prompt**: Gemma converts the verdict + trail into plain, non-technical language for SME owners. 5. **Reviewer feedback loop**: When a reviewer confirms/overturns a verdict, feedback is stored in the case record and referenced in future prompts for similar cases within the session (lightweight session-state adaptation, not persistent model fine-tuning). 6. **Report generator prompt**: Aggregates all confirmed-risk cases into a structured, audit-ready report.

Memory: Session-scoped only for the hackathon (stored in SQLite reviewer_feedback column, referenced in subsequent prompts) - not persistent cross-session memory (that is Tier 3 roadmap: Compliance Memory).

Prompt Template Example (Risk Replay):

```
You are a financial compliance investigator. Given the following flagged transaction data: {data}
Produce a numbered step-by-step reasoning trail explaining why this transaction was flagged, ending with a Risk Score (LOW,
```

Future AI Improvements (Roadmap): Cross-document contradiction detection, persistent Compliance Memory, Reverse Audit (predictive fraud location), Investigator Behavior Cloning.

10. Frontend Architecture

- **Pages:** LoginPage, DashboardPage (triage queue + case list), CaseDetailPage (reasoning trail, verdict, explanation), ReportPage (report preview + export)
- **Routing:** React Router, routes defined in AppRoutes.jsx
- **Components:** Common (Button, Card, Badge, Loader, EmptyState) + feature-specific (CaseList, RiskReplayTrail, ProsecutorVerdict, ExplanationPanel, ReportPreview)
- **State Management:** React Context API (AppContext.js) - sufficient for hackathon scope, no Redux needed
- **Navigation:** Top nav bar with Dashboard / Reports links
- **Responsive Layout:** Mobile-first CSS, dashboard collapses to stacked cards below 768px

11. Backend Architecture

- **Controllers:** Handle request/response logic (auth.controller.js, case.controller.js, report.controller.js)
- **Routes:** Map HTTP endpoints to controllers
- **Services:** Business logic (detection.service.js runs rule engine; gemmaClient.service.js calls Python AI service via internal REST)
- **Middleware:** auth.middleware.js (JWT check), errorHandler.middleware.js (centralized error responses), validate.middleware.js (input validation)
- **Logging:** Simple logger.js utility (console + file for hackathon; swap for Winston/Pino post-hackathon)
- **AI Communication:** Node backend calls Python AI service via axios POST to internal endpoints (e.g., http://localhost:5001/risk-replay)
- **Database Communication:** sqlite3 or better-sqlite3 npm package, queries wrapped in model files

12. Development Roadmap (Phases)

- **Phase 1:** Project setup - repos, folder structure, base configs, .env
- **Phase 2:** Database schema + mock dataset seeding
- **Phase 3:** Detection engine (split invoice + ghost vendor rules) + API endpoints
- **Phase 4:** Python AI service scaffolding + Gemma API connection
- **Phase 5:** AI Risk Replay + AI Prosecutor prompt integration
- **Phase 6:** Frontend dashboard + triage queue + case detail view
- **Phase 7:** Plain-language explanation integration
- **Phase 8:** Audit-ready report generator (backend + frontend)
- **Phase 9:** Integration testing, bug fixes, polish
- **Phase 10:** (Stretch) Timeline, Risk Graph, GST Verification, Vendor Profiling
- **Phase 11:** Demo rehearsal, pitch prep

13. Parallel Development Plan

Developer 1 - Tejash

Owns: AI, Backend, Database, Risk Engine, AI Reasoning, Investigation System, API Development, Python Services

Modules: - backend/src/services/detection.service.js - backend/src/services/riskScoring.service.js - backend/src/services/gemmaClient.service.js - ai-service/ (entire Python microservice) - database/schema.sql, database/seed_data.sql - backend/src/routes/, backend/src/controllers/ - backend/src/models/*

Developer 2 - Yeshwanth

Owns: React Frontend, UI, Dashboard, Components, Visualizations, API Integration, User Experience

Modules: - frontend/src/pages/ - frontend/src/components/ - frontend/src/routes/AppRoutes.jsx - frontend/src/services/apiClient.js - frontend/src/state/AppContext.js - Responsive styling, empty/loading/error states

Shared (both must agree before building): - API contracts (Section 7) - locked before either starts coding. - Case JSON shape (Section 8 schema) - locked before either starts coding. - Naming conventions (Section 15).

Minimal Merge Conflict Strategy: Since Tejash owns backend/ + ai-service/ + database/ and Yeshwanth owns frontend/ entirely, there is close to zero file overlap. The only shared file is docs/API_CONTRACTS.md, which should be finalized in the first 30 minutes before independent work begins.

---

14. Integration Strategy

Branch Strategy: - main - stable, demo-ready branch - dev-tejash - backend/AI work - dev-yeshwanth - frontend work - Merge both into main at defined checkpoints, not continuously

Merge Order: 1. Tejash merges backend scaffolding + mock API responses (even before real logic works, stub endpoints matching the contract) - **first 1 hour**. 2. Yeshwanth builds frontend against these stubs immediately, in parallel. 3. Tejash replaces stubs with real detection/AI logic - Yeshwanth's frontend continues working since the contract shape doesn't change. 4. Final merge: full integration test before demo.

API Contracts: Locked in docs/API_CONTRACTS.md before coding starts - this is the single source of truth both developers reference.

Shared Interfaces: The case JSON object shape (Section 8) is the critical shared interface - any change must be communicated immediately to both developers.

Environment Variables (.env.example):

```
GEMMA_API_KEY=
GEMMA_API_URL=
PORT=5000
AI_SERVICE_URL=http://localhost:5001
DB_PATH=./database/ghostaudit.db
JWT_SECRET=
```

Testing Before Merge: Each developer manually verifies their own module against the API contract using Postman/curl before merging into main.

Final Integration Checklist: - All API endpoints return contract-shaped responses - Frontend renders real (not stubbed) data end-to-end - Gemma calls succeed with real API key - Report export works - No console errors on any page - Mobile responsive check passed

---

15. Coding Standards

- **Naming Conventions:** camelCase for JS variables/functions, PascalCase for React components, snake_case for SQL columns
- **Folder Naming:** lowercase-hyphenated (e.g., ai-service, not AIService)
- **File Naming:** .controller.js, .service.js, .routes.js, .model.js suffixes for backend; PascalCase.jsx for React components
- **Component Naming:** Descriptive, feature-prefixed (e.g., CaseDetailPanel, not Panel1)
- **API Naming:** RESTful, plural nouns (/api/cases, not /api/getCase)
- **Database Naming:** snake_case table/column names, plural table names (transactions, vendors)
- **Git Commit Messages:** type(scope): message - e.g., feat(detection): add split invoice rule logic
- **Branch Naming:** dev-name, feature/short-description

- **Error Handling:** All backend errors pass through errorHandler.middleware.js, consistent JSON error shape
- **Code Formatting:** Prettier + ESLint (Airbnb base config recommended), 2-space indentation

---

16. Milestone Checklist

- [ ] Project scaffolding (frontend + backend + ai-service)
- [ ] Database schema + seed data
- [ ] Split Invoice / Round-Tripping Detection
- [ ] Ghost Vendor Verification
- [ ] Risk scoring API
- [ ] Gemma API connection (ai-service)
- [ ] AI Risk Replay
- [ ] AI Prosecutor (Evidence/Defense/Verdict)
- [ ] Reviewer Triage Queue (frontend + backend)
- [ ] Plain-Language Explanations
- [ ] Audit-Ready Report Generator
- [ ] End-to-end integration test
- [ ] (Stretch) Timeline Investigation
- [ ] (Stretch) Financial Crime Storyboard
- [ ] (Stretch) Risk / Relationship Graph
- [ ] (Stretch) GST Registry Cross-Verification
- [ ] (Stretch) Vendor Risk Profiling
- [ ] Demo rehearsal + pitch prep

---

17. Future Scope (Post-Hackathon)

Cross-Document Contradiction Detection, Compliance Memory, Reverse Audit, Cross-Document Investigation Engine, Investigator Behavior Cloning, Compliance DNA, Peer Benchmark Anomaly Detection, Compliance Drift Detection, Financial Weather Forecast, Invisible Employee, Behavioral Pattern Monitoring, Financial Digital Twin, Behavioral Compliance Intelligence Engine, Multi-Regulator Correlation, Multi-Source Evidence Correlation, Interactive Investigation Workspace, AI Financial Investigation Workspace, SME Self-Audit Copilot (standalone), Delayed Payment Risk & Recovery Agent, DPDP Readiness Checker, MSME Formalization Coach, Regulatory Shock Simulator, Advertising Compliance Auditor, Dark Pattern Financial App Auditor, Property/Collateral Loan Auditor, Cross-Border Customs/HSN Compliance, WhatsApp/Email Financial Compliance, Verbal vs Written Contract Auditor, Shadow Procurement Detector, Auto-Remediation Agent, AI Negotiator, Regional Language Compliance Support.

---

18. Deployment Plan

- **Frontend:** Deploy static build to Vercel/Netlify (React build output)
- **Backend:** Deploy Node/Express API to Render/Railway (free tier sufficient for demo)
- **Python AI Service:** Deploy as a separate Render/Railway service or run locally during demo if API rate limits are a concern
- **Database:** SQLite file bundled with backend deployment (sufficient for hackathon; migrate to Postgres for production)
- **Environment Variables:** Set via hosting platform's dashboard (Gemma API key, JWT secret, service URLs)
- **Production Architecture (future):** Replace SQLite with Postgres, containerize AI service with Docker, add API gateway/rate limiting

---

19. Testing Strategy

- **Frontend Testing:** Manual QA per page (desktop 1280px+, mobile 375px), check loading/empty/error states
- **Backend Testing:** Postman collection covering all endpoints in Section 7, verify success + error responses
- **API Testing:** Contract validation - response shape matches docs/API_CONTRACTS.md exactly
- **AI Testing:** Manually verify Gemma outputs for 3-5 sample cases (split invoice, ghost vendor) produce coherent, non-generic reasoning trails and verdicts
- **Integration Testing:** Full case flow - detection -> risk replay -> verdict -> explanation -> report - run end-to-end without errors
- **Manual Testing:** Full demo walkthrough rehearsed at least twice before presenting to judges

---

20. Final Build Sequence (No Developer Blocked)

1. **[Both, 0-30 min]** Agree on API contracts + case JSON shape -> write docs/API_CONTRACTS.md
2. **[Tejash, 30 min-1.5 hr]** Scaffold backend + stub endpoints returning mock JSON matching the contract
3. **[Yeshwanth, 30 min-1.5 hr, parallel]** Scaffold frontend, build pages/components against Tejash's stubs
4. **[Tejash, 1.5-3 hr]** Build database schema, seed mock data, implement real detection engine (split invoice + ghost vendor)
5. **[Yeshwanth, 1.5-3 hr, parallel]** Build dashboard + triage queue UI, wire to stub API, polish loading/empty states
6. **[Tejash, 3-5 hr]** Build Python AI service, integrate Gemma for Risk Replay + Prosecutor + Explanation prompts
7. **[Yeshwanth, 3-5 hr, parallel]** Build case detail page (reasoning trail, verdict, explanation panels), wire to real endpoints as they come online
8. **[Tejash, 5-6 hr]** Build report generation endpoint + Gemma report prompt
9. **[Yeshwanth, 5-6 hr, parallel]** Build report preview + export page
10. **[Both, 6-7 hr]** Full integration - replace all stubs with real data, fix mismatches
11. **[Both, 7-8 hr]** End-to-end testing, bug fixes, UI polish
12. **[Both, 8+ hr]** Stretch features (Timeline, Risk Graph, GST Verification) only if ahead of schedule
13. **[Both, final]** Demo rehearsal, pitch narrative alignment, submission

---

21. Feature Ownership Split - Tejash vs Yeshwanth

This maps every Tier 1 and Tier 2 feature (plus key Tier 3 roadmap items) to the exact developer responsible, so there's zero ambiguity on demo day.

Tejash - AI, Backend, Database, Risk Engine, Investigation Logic

- **Split Invoice / Round-Tripping Detection:** Rule logic in detection.service.js, threshold-breach algorithm, API endpoint /api/cases/scan
- **Ghost Vendor Verification:** GSTIN format/address/bank-pattern validation logic, vendor scoring in detection.service.js
- **AI Risk Replay:** risk_replay_prompt.py, Gemma API call in ai-service/, response parsing into step-by-step JSON
- **AI Prosecutor (Evidence -> Defense -> Verdict):** prosecutor_prompt.py, verdict generation pipeline, /api/cases/:id/verdict endpoint
- **Reviewer Triage Queue (backend logic):** Sorting/ranking logic by severity + confidence in case.controller.js, /api/cases endpoint with sort params
- **Plain-Language Explanations (AI side):** explain_prompt.py, Gemma call + response formatting, /api/cases/:id/explain endpoint
- **Audit-Ready Report Generator (backend/AI side):** report_generator.py, report.service.js, /api/reports/generate and /api/reports/:id/export endpoints
- **Timeline Investigation (stretch):** Backend logic to reconstruct event sequence from transaction timestamps

- **Risk / Relationship Graph (stretch, data side):** Backend endpoint producing graph-ready JSON (nodes = vendors/accounts, edges = transactions)
- **GST Registry Cross-Verification (stretch):** Mock GST registry lookup logic, verification endpoint
- **Vendor Risk Profiling (stretch):** Aggregated vendor scoring logic across multiple flagged cases
- **Database schema + mock dataset:** database/schema.sql, database/seed_data.sql, SQLite setup
- **Auth:** auth.controller.js, auth.middleware.js, JWT logic
- **Reviewer feedback loop:** /api/cases/:id/feedback endpoint, session-state adaptation logic

Yeshwanth - React Frontend, UI, Dashboard, Visualizations, UX

- **AI Risk Replay (display):** RiskReplayTrail component - renders step-by-step reasoning visually
- **AI Prosecutor (display):** ProsecutorVerdict component - Evidence/Defense/Verdict panel layout
- **Reviewer Triage Queue (UI):** TriageQueue and CaseList components - sortable, filterable case list on DashboardPage
- **Plain-Language Explanations (display):** ExplanationPanel component - renders Gemma's plain-language text clearly for non-experts
- **Audit-Ready Report Generator (UI):** ReportPreview and ExportButton components on ReportPage
- **Timeline Investigation (stretch, UI):** Timeline visualization component (horizontal event chain)
- **Financial Crime Storyboard (stretch):** Narrative-style card sequence component presenting the case story
- **Risk / Relationship Graph (stretch, UI):** Graph visualization (lightweight JS graph library) rendering backend graph JSON
- **GST Registry Cross-Verification (stretch, UI):** Verification status badge/indicator on vendor cards
- **Vendor Risk Profiling (stretch, UI):** Vendor trust score badge/visual on vendor detail views
- **Login/Auth UI:** LoginPage, session handling in AppContext.js
- **Dashboard shell + navigation:** AppRoutes.jsx, nav bar, responsive layout
- **Loading/empty/error states:** Skeleton loaders, empty-state illustrations, error banners across all pages
- **API integration:** apiClient.js - wiring every backend endpoint into the frontend

Shared / Collaborative (Both Must Align)

- **API Contracts:** Locked together before coding starts.
- **Case JSON Shape:** Both reference the same schema - any change requires mutual sign-off.
- **Demo Narrative:** Both rehearse the final pitch flow together, even though each owns different technical parts.

Quick Summary

- **Tejash = everything behind the API** (detection logic, Gemma prompts, scoring, database, report generation logic).
- **Yeshwanth = everything the reviewer/user sees and clicks** (dashboard, case views, visual components, report display, UX polish).
- Neither developer touches the other's folder (backend + ai-service + database vs. frontend), which keeps merge conflicts near zero.